Variables and Constants

Hello, World!

We are going to do an important tradition of programming, whenever you start learning a new language, the first thing you do is write a "Hello, World!" program. So let's write one of those up.

```
print("Hello, world!")
```

Comments

Now that we have started our coding journey properly, I want to take a quick sidebar to talk about a useful tool, comments. Comments allow us as programmers to write regular human readable text inside of our code files. This can be great for taking notes while going through this course! Comments are awesome for leaving little notes about why you made a decision, or describing something as it happens. They also can be used when you need to figure out a problem by plotting out steps.

Swift supports two different kinds of comments, single, and multiline. Single line are the most common in my experience, and we can write them by prefacing any line with `//`. Then anything after the `//` on that line of code will be ignored, and isn't included in our code anymore.

```
// Our first program
print("Hello, world!") // Prints Hello, world!
```

You can also do multiline comments. Technically speaking these do not have to span multiple lines, but they typically do. Instead of only marking the start of a comment, like with single line, you mark both the start and end of the comment. You use `/*` & `*/` to mark the start and end of a multiline comment respectively.

```
/*
This is a comment
It spans as many lines as I want
In this case it spans 3
Just kidding :P
*/
```

Variables

```
var greeting = "Hello, playground"
```

Upon opening a new playground, you will be greeted with one simple line of code, depicted above. This line of code is creating a variable, which is a way that we can store data to be used later. There are four main pieces to this syntax:

1. `var` : This is a keyword that is basically saying: “Create a new variable”
2. `greeting` : This is what you are naming your variable, it can be almost anything you want it to be; however, you normally want it to describe what it is representing.
3. `The = Sign` : This is what is assigning the value to our newly created and named variable.
4. `"Hello, playground"` : This is the value you are setting to the variable. Notice the `""` quotation marks on either side of the words. This is so Swift knows where the text we are writing starts and ends, but we will get more into that later on.

Now that you know the basic structure of a variable, let's have you write your very first one!

```
var myFirstVariable = "Hello, world!"
```

Variables are called variables because they are subject to change. So if I want, I can choose to change what data is stored when I reference the name of my variable.

```
var name = "Finn"
name = "Jake"
name = "Simon"
```

When you are declaring variables, you can actually declare multiple of them on the same line; however, personally I avoid this as I think it can be very confusing.

```
var x = 0, y = 2, z = 1
```

Constants

Having data that can be changed is very useful, and is an important part of programming; however, equally useful, is being able to have data that cannot be changed once it has been set. That might seem a little weird when you are just starting out, but trust me, this is a very significant piece of information.

Luckily for us, there is a built-in way to create these immutable pieces of data. They are called constants instead of variables, and the syntax is very similar to how you create a variable. All you need to do is swap the keyword used. Instead of using the keyword `var` you use the keyword `let` by using this keyword, we are creating a constant. A constant, as you may have inferred, stores a piece of data, that you can reference later but cannot change.

Let's create our first constant, shall we?

```
let answer = 42

// If we try and change what value is stored
// The compiler will throw an error, and your program will not run
// ERROR: Cannot assign to value: `answer` is a `let` constant
answer = 10
// Comment out the above line using
```

Simple Types

Strings

Now that we understand the basic idea of storing small bits of data, it's time to learn about what we can store in them. So far, we have been primarily storing what are called `Strings`. A `String` is essentially just text. We create strings using `""` quotation marks surrounding the value of our `String`. Strings are one of the most basic values you can use, meaning you will likely use them a lot.

```
var person = "Ryan Reynolds"
```

Multiline Strings

There is a second form of `String`. That as far as Swift cares is the same thing, but when it comes to how we write out the string, there are a couple of key differences. Firstly being that when you create a normal string, it will always be on a single line in the code editor. This means that if you have a very long string to write out, such as a paragraph, it can be difficult to write. Multiline Strings have default support for multiple lines, and let us write out our content on multiple lines.

To create a multiline string, you surround your text with `""" """` triple quotation marks, where each of the sets of quotations is on its own line.

Let's say for example, I want to display a haiku to my user, using a multiline string it's super easy, we could do it like this:

```
var myHaiku = """
Coding is the best
Don’t let it make you feel dumb
Keep calm, debug on!
"""
```

By default, multiline strings will output the exact same way that it is input, including all its new-line characters. However, sometimes you may want to format a long string, but without introducing newline characters, you can do that by adding the `\` to signify that this is not the end of a line.

```
myHaiku = """
Coding is the best \
Don't let it make you feel dumb \
Keep calm, debug on!
"""
```

Integers

Let's be honest here, text is really cool and all, but it's rather limited. Besides, computers are built on numbers and math, not letters.

There are many forms of numbers, but we are going to start by talking about two main types. Whole and decimal numbers. Just like we are back learning math for the first time, we are going to start by talking about whole numbers.

In Swift, we reference whole numbers as `Integer(s)` or `Int(s)`. Let's go ahead and create a new constant that will store an `Int` for us.

```
let weAre = 1
```

Unlike with `Strings` we don't have any special syntax like the quotation marks. We just put the number right in. So in the above line we are making a new constant called `weAre` and store the value `1` inside of it. In other words, we are saying that “We are number one.”

`Ints` have a very large value range. On most modern hardware the range is between `-9,223,372,036,854,775,807` & `9,223,372,036,854,775,807`. Or around 9 1/4 Quintilian positive or negative. That is a very high range of numbers, and it can be really

confusing to write those numbers, as we cannot use commas in our number statement. Let's try writing the number one million out in code:

```
var oneMillion = 1000000
```

It's obviously super hard to tell what number that is at a glance, and since we cannot use commas, we need a better way to write these numbers. Luckily, Swift does have a way to help us write out these long numbers, we can use `_` underscores! A common practice with this feature is whenever you would normally put commas, you instead put underscores. Here is an example:

```
oneMillion = 1_000_000 // Much easier to read now!

// Swift will ignore underscores in our number literals.
// So we can put as many or as few as we want!

oneMillion = 1___00_0_______0_0_0
```

As far as Swift is concerned, all three of the ways we wrote the number literal are the exact same.

Now that we can store numbers, you are probably asking yourself what good are numbers if we can only store and look at them? Well, I'm glad you are such a go-getter and wanted to do things with these numbers. So let's move onto everyone's favorite subject, Math!

Swift lets us do many operations on our numbers. We are going to talk about the 4 big guys right now: Addition, Subtraction, Multiplication, & Division.

```
let oneMillionAndOne = oneMillion + 1 // 1_000_001
let oneMillionMinusOne = oneMillion - 1 // 999_999
let twoMillion = oneMillion * 2 // 2_000_000
let halfAMillion = oneMillion / 2 // 500_000
let oneMillionSquared = oneMillion * oneMillion // 1_000_000_000_000
```

Above, we are making new constants for every result, but we don't need to do that. If our original number is a variable, we can update it with the changes example:

```
var myHealth = 100
let damage = 10
myHealth = myHealth - damage
```

Now `myHealth` has a value of 90. We can do the same thing for all the other operators as well. This does lead to some kind of wordy syntax. You will also have to do this kind of

operation a bunch. Luckily when you are mutating a variable with math, there is a shorthand way to do what we just did.

```
var total = 4
total += 8
total -= 2
total *= 10
total /= 5
// Whats the value of total?
```

Each of these operations updates the value using the given operation and the value to change it by. It's essentially doing the `myHealth = myHealth - damage` just in a shorter, cleaner way. So let's look over that code again, but now that you have taken our mini test, let's show how the number changes throughout.

```
var total = 4 // 4
total += 8    // 4 + 8 = 12
total -= 2    // 12 - 2 = 10
total *= 10   // 10 * 10 = 100
total /= 5    // 100 / 5 = 20
// Whats the value of total? The value is 20!
```

On top of the main 4 mathematical operators that you are likely already familiar with, there is one more that is very important, Modulo. Modulo or mod for short is an operator that is used to get the remainder of dividing two whole numbers. Modulo is represented in code by using the `%` operator.

```
let remainder = 3 % 2 // 1
```

Modulo also has support for the shorthand operator, you use it the same as the other ones!

```
var theNumberToGetTheRemainderFrom = 10
theNumberToGetTheRemainderFrom %= 3 // 1
```

Doubles

Whole numbers are cool and all, but what about when we need to use partial numbers?

The main option in Swift for handling decimal numbers is using what is called a `Double`. A `Double` is simply a decimal number. The declaration of a `Double` is exactly what you think it is.

```
var myFirstDecimalNumber = 42.0
```

It is as simple as adding a decimal point to our number. We can do most all the same things to `Doubles` as we can do with `Ints` and in the same way. The one exception, for now, being modulo, as we don't need remainder calculations for decimal numbers.

```
let fourtyThreePointFive = myFirstDecimalNumber + 1.5
myFirstDecimalNumber += 25.5
myFirstDecimalNumber -= 3
myFirstDecimalNumber *= 2
myFirstDecimalNumber /= 3
// Whats the Value of myFirstDecimalNumber?
```

Answer Key:

```
myFirstDecimalNumber += 25.5 // 67.5
myFirstDecimalNumber -= 3 // 64.5
myFirstDecimalNumber *= 2 // 129
myFirstDecimalNumber /= 3 // 43
// The Answer Is 43!
```

Booleans

The next type we are going to talk about are Booleans or `Bools` for short. `Bools` are incredibly useful, yet very simple. A `Bool` is a value that represents either `true` or `false`.

```
let thisIsABool = true
let thisIsNotABool = false
```

`Bools` are very useful, and you will use them all the time, even if you aren't using the `true` or `false` literals directly.

Working With Strings

Now we have covered the most basic types in Swift. There are many more than we have covered, but we will go over them in later lessons. Right now, we are going to talk about different ways we can work with Strings.

String Concatenation

The most basic way to modify `Strings` in Swift is by using what is called **String Concatenation**. **String Concatenation** is just a fancy way of saying: "Combining Strings Together". Let's do an example of **String Concatenation.**

```
let candy = "Peppermint"
let position = "Butler"

// We can add Strings together by using the '+' operator.
var candyPerson = candy + position
```

This is how we can **concat** `Strings` together; however, if we look at `candyPerson` you may notice it may not be what you're expecting. The value that `candyPerson` holds currently is `PeppermintButler`, there is a lack of a space. This is because when we **concat** `Strings` our result is going to simply be the two `Strings` mashed together.

This is something that is easy to remedy, right? We can just add the space between our words manually like this:

```
candyPerson = candy + " " + position
```

This will absolutely work; however, I imagine you could see how tedious and hard to read this would be with a more complex **concatenation**. Luckily for us, Swift has a feature that makes this process a lot easier, **String Interpolation**.

String Interpolation

String Interpolation is the other major way that we can modify `Strings`. It is also the more common way you will see, and you should likely just default to using it in most cases. **String Interpolation** uses a special syntax to insert other values into a `String`. The special syntax is a `\(<VALUE>)` inside a `String` literal.

Let's do an example where we set `candyPerson` to a new value using **String Interpolation**.

```
candyPerson = "\(candy) \(position)"
```

This above syntax gives us the same result as before, but in a much easier to read way. Let's move onto some more examples of **String Interpolation** where we mix more than just

`Strings` so you can understand better.

For all the basic types, **String Interpolation** is incredibly simple, it's essentially the same thing as mixing `Strings` with **String Interpolation**.

```
var penguinNumber = 5
let penguinName = "Gunter"

var penguinGreeting = "Hello, \(penguinName) #\(penguinNumber)!"
// "Hello, Gunter #5!"
```

When we use the `\()` syntax, we are really saying that we want to create a `String` from the content inside, and then combine it into this `String`. You cannot mix `Strings` and Numbers together without making the number into a `String` first or vice versa.

Type Annotation & Type Inference

So far, there has been something going on beneath the hood of Swift this entire time. It's called **Type Inference**. What that means is that thanks to Swift, you haven't had to tell Xcode what types your variables and constants are. As Swift is smart enough to infer it for you. That will not always be the case. When you are doing simple things like what we are doing above, It's easy to figure out what things are supposed to be; however, there may not always be enough information to make an inference. When we have situations like that, we need to use a thing called **Type Annotation**.

For example, let's say I have a Whole number that I need to store, but I need it to be interacted with as if it is a decimal number.

```
var decimalNumber = 1
```

The above example is not a `Double` it is being stored as an `Int` so I didn't solve the problem I had. There are two ways I could fix this problem, I could either replace the `1` with a `1.0` or I could use **Type Annotation**! Let's fix it with **Type Annotation**:

The syntax of **Type Annotation** is simple, when we declare a variable or constant we add a `:` and the Type that we want it to be. So for the above example, we would do this:

```
var actualDecimalNumber: Double = 1
```

Now since we said we only want to store `Doubles` inside this variable, when I store `1` inside of it, it still treats it like a decimal number! We can use **Type Annotation** for any type!

```swift
let annotatedInt: Int = 0
let annotatedDouble: Double = 0
let annotatedString: String = ""
let annotatedBoolean: Bool = false
```